

LAPORAN PRATIKUM
STRUKTUR DATA
“Double LinkedList”



Dosen Pengampu:
DR. Wahyudi. .S.T.M.T

Asisten Lab:
Jovantri Immanuel Gulo

Disusun Oleh:
Marcello Bayu Denar Widjaksono
2511532011

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “LinkedList”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan program komputer, pemahaman mendalam tentang struktur data (data structure) menjadi fondasi utama bagi para programmer. Salah satu bentuk struktur data yang paling sering digunakan adalah Abstract Data Type (ADT), yang berfungsi mengorganisir data agar mudah diakses, dimanipulasi, dan dikelola untuk meningkatkan kinerja aplikasi. Pada bahasa pemrograman Java, implementasi ADT List berbasis tautan dapat dilakukan melalui kelas `LinkedList` yang tersedia dalam paket `java.util`. `LinkedList` merupakan struktur data linear yang terdiri dari rangkaian node (simpul) yang saling terhubung melalui referensi, di mana setiap node menyimpan nilai data serta alamat ke node berikutnya (dan sebelumnya pada implementasi `DoublyLinkedList`). Perbedaan mendasar antara `LinkedList` dengan struktur data linear berbasis indeks seperti `Array` terletak pada alokasi memori dan mekanisme akses elemen: `LinkedList` tidak memerlukan blok memori yang bersebelahan dan ukurannya bersifat dinamis, sehingga operasi penyisipan serta penghapusan elemen menjadi sangat efisien tanpa perlu menggeser data lain, meskipun akses acak memerlukan penelusuran sekuensial dari node awal. Karakteristik ini membuat `LinkedList` sangat fleksibel dan banyak diterapkan dalam berbagai skenario nyata, seperti implementasi riwayat navigasi (browser history), fitur undo/redo pada aplikasi, manajemen playlist, representasi graf melalui adjacency list, serta pembuatan cache dengan algoritma LRU (Least Recently Used).

1.2 Tujuan Pratikum

1. Menjelaskan konsep dan karakteristik `LinkedList` sebagai implementasi struktur data ADT berbasis node yang terhubung secara linear dalam Java.
2. Mengidentifikasi metode-metode utama `LinkedList` (seperti `add`, `remove`, `get`, `set`, `peek`, dan `size`) beserta mekanisme kerjanya.

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman konsep struktur data dinamis dan non-kontigu melalui penggunaan LinkedList dalam bahasa pemrograman Java.
2. Mampu menggunakan metode-metode utama LinkedList untuk menyelesaikan permasalahan komputasi yang relevan..

BAB II PEMBAHASAN

2.1 Pratikum Striktur Data

```
package pekan6_2511532011;

public class NodeDLL_2511532011 {
    int data_2011;
    NodeDLL_2511532011 next_2011;
    NodeDLL_2511532011 prev_2011;

    public NodeDLL_2511532011(int data_2011) {
        this.data_2011 = data_2011;
        this.next_2011 = null;
        this.prev_2011 = null;
    }
}
```

Pembahasan Class NodeDLL:

1. Deklarasi Kelas

`public class NodeDLL_2511532011`: Mendefinisikan kelas publik yang merepresentasikan satu simpul (node) dalam Doubly Linked List.

Kelas ini bersifat public agar dapat diakses oleh kelas lain seperti kelas driver atau kelas Doubly Linked List utama.

2. Atribut Data (`data_2011`)

`int data_2011`:: Deklarasi variabel instance bertipe integer untuk menyimpan nilai/data yang dibawa oleh node ini.

Dalam konteks DLL, atribut ini adalah payload atau muatan informasi yang ingin disimpan dalam struktur data.

Akses modifier default (`package-private`) memungkinkan kelas lain dalam paket yang sama untuk mengaksesnya secara langsung.

3. Atribut Pointer Next (`next_2011`)

`NodeDLL_2511532011 next_2011`:: Deklarasi variabel instance bertipe referensi ke kelas `NodeDLL_2511532011` itu sendiri.

Berfungsi sebagai pointer forward yang menghubungkan node saat ini ke node berikutnya dalam rangkaian linked list.

Mirip dengan Singly Linked List, pointer ini memungkinkan traversing dari depan ke belakang.

4. Atribut Pointer Prev (prev_2011)

NodeDLL_2511532011 prev_2011;; Deklarasi variabel instance bertipe referensi ke kelas NodeDLL_2511532011 itu sendiri.

Perbedaan Utama dengan SLL: Pointer prev memungkinkan navigasi mundur, yaitu dari node saat ini ke node sebelumnya.

Fitur ini memberikan fleksibilitas lebih: traversing dua arah (bidirectional), penghapusan node lebih efisien jika referensinya sudah diketahui, dan operasi insert lebih fleksibel.

5. Konsep Self-Referential Class

Kelas NodeDLL_2511532011 memiliki atribut yang bertipe sama dengan kelas itu sendiri (next_2011 dan prev_2011).

Konsep ini disebut self-referential class, yang menjadi dasar pembentukan struktur data berantai seperti linked list.

Setiap node "mengenal" tetangganya di kedua sisi, membentuk rantai dua arah.

6. Konstruktor Node

```
public NodeDLL_2511532011(int data_2011);
```

 Mendefinisikan konstruktor publik yang menerima satu parameter integer untuk inisialisasi node.

Konstruktor ini dipanggil setiap kali ingin membuat node baru dalam doubly linked list.

7. Inisialisasi Data dalam Konstruktor

```
this.data_2011 = data_2011;;
```

 Mengassign nilai parameter data_2011 ke variabel instance this.data_2011.

Keyword this digunakan untuk membedakan antara atribut kelas dengan parameter metode yang memiliki nama sama.

8. Inisialisasi Pointer Next

```
this.next_2011 = null;;
```

 Mengassign nilai null ke atribut pointer next_2011.

Inisialisasi null menandakan bahwa node baru ini belum terhubung ke node berikutnya.

Dalam DLL, node dengan next = null biasanya merepresentasikan node terakhir (tail).

9. Inisialisasi Pointer Prev

this.prev_2011 = null;; Mengassign nilai null ke atribut pointer prev_2011.

Inisialisasi null menandakan bahwa node baru ini belum terhubung ke node sebelumnya.

Dalam DLL, node dengan prev = null biasanya merepresentasikan node pertama (head).


```

package pekan6_2511532011;

public class InsertDLL_2511532011 {

    static NodeDLL_2511532011 insertBegin(NodeDLL_2511532011 head_2011, int data_2011) {

        NodeDLL_2511532011 new_node_2011 = new NodeDLL_2511532011(data_2011);

        new_node_2011.next_2011 = head_2011;

        if (head_2011 != null) {

            head_2011.prev_2011 = new_node_2011;

        }

        return new_node_2011;

    }

    public static NodeDLL_2511532011 insertEnd (NodeDLL_2511532011 head_2011, int newData_2011) {

        NodeDLL_2511532011 newNode_2011 = new NodeDLL_2511532011(newData_2011);

        if (head_2011 == null) {

            head_2011 = newNode_2011;

        }else {

            NodeDLL_2511532011 curr_2011 = head_2011;

            while (curr_2011.next_2011 != null) {

                curr_2011 = curr_2011.next_2011;

            }

            curr_2011.next_2011 = newNode_2011;

            newNode_2011.prev_2011 = curr_2011;

        }

        return head_2011;

    }

    public static NodeDLL_2511532011 insertAtPosition (NodeDLL_2511532011 head_2011, int pos_2011, int new_data_2011) {

        NodeDLL_2511532011 new_node_2011 = new NodeDLL_2511532011(new_data_2011);

        if (pos_2011 == 1) {

            new_node_2011.next_2011 = head_2011;

            if (head_2011 != null) {

                head_2011.prev_2011 = new_node_2011;

            }

            head_2011 = new_node_2011;

            return head_2011;

        }

        NodeDLL_2511532011 curr_2011 = head_2011;

        for (int i = 1; i < pos_2011 - 1 && curr_2011 != null; ++i) {

            curr_2011 = curr_2011.next_2011;

        }

        if (curr_2011 == null) {

            System.out.println("Posisi tidak ada");

            return head_2011;

        }

        new_node_2011.prev_2011 = curr_2011;

        new_node_2011.next_2011 = curr_2011.next_2011;

        curr_2011.next_2011 = new_node_2011;

        if (new_node_2011.next_2011 != null) {

            new_node_2011.next_2011.prev_2011 = new_node_2011;

        }

        return head_2011;

    }

}

```

```

public static void printList (NodeDLL_2511532011 head_2011) {
    NodeDLL_2511532011 curr_2011 = head_2011;
    while (curr_2011 != null) {
        System.out.print(curr_2011.data_2011 + " <-> ");
        curr_2011 = curr_2011.next_2011;
    }
    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(2);
    head_2011.next_2011 = new NodeDLL_2511532011 (3);
    head_2011.next_2011.prev_2011 = head_2011;
    head_2011.next_2011.next_2011 = new NodeDLL_2511532011(5);
    head_2011.next_2011.next_2011.prev_2011 = head_2011.next_2011;
    System.out.print("DLL awal: ");
    printList (head_2011);
    head_2011 = insertBegin (head_2011, 1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList(head_2011);
    System.out.print("simpul 6 ditambah di akhir: ");
    int data_2011 = 6;
    head_2011 = insertEnd( head_2011, data_2011);
    printList(head_2011);
    System.out.print("tambah node 4 di posisi 4:");
    int data2_2011 = 4;
    int pos_2011 = 4;
    head_2011 = insertAtPosition(head_2011, pos_2011, data2_2011);
    printList(head_2011);
}
}

```

Pembahasan Class InsertDLL:

1. Deklarasi Kelas

public class InsertDLL_2511532011: Kelas publik yang berisi kumpulan metode statis untuk operasi penyisipan pada Doubly Linked List.

Kelas ini berfungsi sebagai utility class yang menyediakan metode insert di awal, akhir, dan posisi tertentu dengan manipulasi pointer dua arah.

2. Metode insertBegin (Sisip di Awal)

static NodeDLL_2511532011 insertBegin(NodeDLL_2511532011 head_2011, int data_2011): Metode statis untuk menambahkan node baru di posisi paling depan (head) DLL.

NodeDLL_2511532011 new_node_2011 = new NodeDLL_2511532011(data_2011);: Membuat instance node baru dengan nilai yang diberikan.

new_node_2011.next_2011 = head_2011;: Logika Forward Link. Pointer next dari node baru diarahkan ke head lama.

if (head_2011 != null) { head_2011.prev_2011 = new_node_2011; }: Logika Backward Link (Khas DLL). Jika list tidak kosong, pointer prev dari head lama diarahkan kembali ke node baru. Ini adalah perbedaan utama dengan SLL.

return new_node_2011;: Mengembalikan referensi node baru sebagai head yang updated.

Kompleksitas: $O(1)$ karena tidak perlu traversing.

3. Metode insertEnd (Sisip di Akhir)

public static NodeDLL_2511532011 insertEnd(...): Metode statis untuk menambahkan node baru di posisi paling akhir (tail) DLL.

NodeDLL_2511532011 newNode_2011 = new NodeDLL_2511532011(newData_2011);: Membuat node baru dengan nilai yang diberikan.

if (head_2011 == null) { head_2011 = newNode_2011; }: Handle kasus list kosong, node baru langsung menjadi head (dan juga tail).

NodeDLL_2511532011 curr_2011 = head_2011;: Inisialisasi pointer curr untuk traversing dari head.

while (curr_2011.next_2011 != null) { curr_2011 = curr_2011.next_2011; }: Loop traversing hingga menemukan node terakhir (tail) dimana next bernilai null.

curr_2011.next_2011 = newNode_2011;: Logika Forward Link. Hubungkan tail saat ini ke node baru.

`newNode_2011.prev_2011 = curr_2011;;` Logika Backward Link. Hubungkan pointer prev node baru ke tail lama. Ini memastikan koneksi dua arah tetap utuh.

`return head_2011;;` Mengembalikan head asli karena posisi head tidak berubah.

Kompleksitas: $O(n)$ karena harus traversing seluruh list untuk mencapai tail.

4. Metode `insertAtPosition` (Sisip di Posisi Tertentu)

`public static NodeDLL_2511532011 insertAtPosition(...):` Metode untuk menyisipkan node pada posisi index tertentu (1-based index).

`NodeDLL_2511532011 new_node_2011 = new NodeDLL_2511532011(new_data_2011);` Membuat node baru dengan nilai target.

`if (pos_2011 == 1) { ... }` Kasus khusus jika posisi = 1, delegasikan logika ke `insertBegin` (sama dengan head insertion).

`NodeDLL_2511532011 curr_2011 = head_2011;;` Inisialisasi pointer untuk traversing menuju posisi sebelum target.

`for (int i = 1; i < pos_2011 - 1 && curr_2011 != null; ++i) { curr_2011 = curr_2011.next_2011; }` Loop untuk bergerak hingga mencapai node di posisi pos-1 (node sebelum posisi target).

`if (curr_2011 == null) { System.out.println("Posisi tidak ada"); return head_2011; }` Validasi jika posisi melebihi panjang list.

Empat Langkah Penyambungan (Khas DLL):

`new_node_2011.prev_2011 = curr_2011;;` Node baru menunjuk ke belakang (prev) ke node saat ini.

`new_node_2011.next_2011 = curr_2011.next_2011;;` Node baru menunjuk ke depan (next) ke node setelah posisi target.

`curr_2011.next_2011 = new_node_2011;;` Node saat ini sekarang menunjuk ke depan ke node baru.

`if (new_node_2011.next_2011 != null) { new_node_2011.next_2011.prev_2011 = new_node_2011; }` Penting:

Jika ada node setelah node baru, update pointer prev node tersebut agar menunjuk kembali ke node baru. Ini menjaga konsistensi link dua arah.
return head_2011;; Kembalikan referensi head.

5. Metode printList (Cetak List)

public static void printList(NodeDLL_2511532011 head_2011): Metode untuk menampilkan isi DLL ke konsol.

NodeDLL_2511532011 curr_2011 = head_2011;; Inisialisasi pointer curr untuk traversing mulai dari head.

while (curr_2011 != null) { System.out.print(curr_2011.data_2011 + " <-> "); ... }: Loop mencetak setiap nilai data diikuti pemisah " <-> " untuk memvisualisasikan koneksi dua arah.

System.out.println(); Pindah baris setelah selesai mencetak. Output akan terlihat seperti: 1 <-> 2 <-> 3 <->.

6. Metode main - Inisialisasi DLL Awal

NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(2);; Membuat node pertama dengan nilai 2 sebagai head.

head_2011.next_2011 = new NodeDLL_2511532011(3);; Menambahkan node kedua dengan nilai 3.

head_2011.next_2011.prev_2011 = head_2011;; Penting: Menghubungkan pointer prev node 3 kembali ke node 2. Ini membangun link dua arah.

head_2011.next_2011.next_2011 = new NodeDLL_2511532011(5);; Menambahkan node ketiga dengan nilai 5.

head_2011.next_2011.next_2011.prev_2011 = head_2011.next_2011;; Menghubungkan pointer prev node 5 kembali ke node 3.

State Awal: DLL terbentuk: null <- [2] <-> [3] <-> [5] -> null.

7. Metode main - Cetak DLL Awal

System.out.print("DLL awal: ");; Label output.

printList(head_2011);; Memanggil metode print untuk menampilkan: 2 <-> 3 <-> 5 <->.

8. Metode main - Insert Begin (Nilai 1)

head_2011 = insertBegin(head_2011, 1);; Panggil metode insert di awal.

Proses: Node 1 dibuat, 1.next = 2, 2.prev = 1, return node 1 sebagai head baru.

State: null <- [1] <-> [2] <-> [3] <-> [5] -> null.

printList(head_2011);: Cetak hasil: 1 <-> 2 <-> 3 <-> 5 <->.

9. Metode main - Insert End (Nilai 6)

int data_2011 = 6; head_2011 = insertEnd(head_2011, data_2011);:

Panggil metode insert di akhir.

Proses: Traversing ke tail (5), lalu 5.next = 6, 6.prev = 5.

State: null <- [1] <-> [2] <-> [3] <-> [5] <-> [6] -> null.

printList(head_2011);: Cetak hasil: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->.

10. Metode main - Insert at Position 4 (Nilai 4)

int data2_2011 = 4; int pos_2011 = 4;: Siapkan nilai 4 untuk disisipkan di posisi ke-4.

head_2011 = insertAtPosition(head_2011, pos_2011, data2_2011);:

Sisipkan node.

Proses: Traversing ke posisi 3 (node 3), lalu lakukan 4 langkah penyambungan:

4.prev = 3

4.next = 5 (karena 3.next adalah 5)

3.next = 4

5.prev = 4 (karena 4.next adalah 5)

State Akhir: null <- [1] <-> [2] <-> [3] <-> [4] <-> [5] <-> [6] -> null

```

package pekan6_2511532011;

public class HapusDLL_2511532011 {

    public static NodeDLL_2511532011 delHead_2011(NodeDLL_2511532011 head_2011) {

        if (head_2011 == null) {

            return null; }

        NodeDLL_2511532011 temp_2011 = head_2011;

        head_2011 = head_2011.next_2011;

        if (head_2011 != null) {

            head_2011.prev_2011 = null;}

        return head_2011;

    }

    public static NodeDLL_2511532011 dellast (NodeDLL_2511532011 head_2011) {

        if (head_2011 == null) {

            return null; }

        if (head_2011.next_2011 == null) {

            return null; }

        NodeDLL_2511532011 curr_2011 = head_2011;

        while (curr_2011.next_2011 != null) {

            curr_2011 = curr_2011.next_2011;

        }

        if (curr_2011.prev_2011 != null) {

            curr_2011.prev_2011.next_2011 = null;}

        return head_2011;

    }

    public static NodeDLL_2511532011 delPos_2011(NodeDLL_2511532011 head_2011, int pos_2011) {

        if (head_2011 == null) {

            return head_2011;}

        NodeDLL_2511532011 curr_2011 = head_2011;

        for (int i = 1; curr_2011 != null && i < pos_2011; ++i) {

            curr_2011 = curr_2011.next_2011;}

        if (curr_2011 == null) {

            return head_2011;}

        if (curr_2011.prev_2011 != null) {

            curr_2011.prev_2011.next_2011 = curr_2011.next_2011;}

        if (curr_2011.next_2011 != null) {

            curr_2011.next_2011.prev_2011 = curr_2011.prev_2011;}

        if (head_2011 == curr_2011) {

            head_2011 = curr_2011.next_2011;}

        return head_2011;

    }

    public static void printList (NodeDLL_2511532011 head_2011) {

        NodeDLL_2511532011 curr_2011 = head_2011;

        while (curr_2011 != null) {

            System.out.print(curr_2011.data_2011 + " ");

            curr_2011 = curr_2011.next_2011;

```

```

    }

    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(1);
    head_2011.next_2011 = new NodeDLL_2511532011(2);
    head_2011.next_2011.prev_2011 = head_2011;
    head_2011.next_2011.next_2011 = new NodeDLL_2511532011(3);
    head_2011.next_2011.next_2011.prev_2011 = head_2011.next_2011;
    head_2011.next_2011.next_2011.next_2011 = new NodeDLL_2511532011(4);
    head_2011.next_2011.next_2011.next_2011.prev_2011 = head_2011.next_2011.next_2011;
    head_2011.next_2011.next_2011.next_2011.next_2011 = new NodeDLL_2511532011(5);
    head_2011.next_2011.next_2011.next_2011.next_2011.prev_2011 = head_2011.next_2011.next_2011.next_2011;

    System.out.print("DLL Awal: ");
    printList(head_2011);
    System.out.print("Setelah head dihapus: ");
    head_2011 = delHead_2011(head_2011);
    printList(head_2011);
    System.out.print("Setelah node terakhir dihapus: ");
    head_2011 = delHead_2011(head_2011);
    printList(head_2011);
    System.out.print("menghapus node terakhir dihapus: ");
    head_2011 = delHead_2011(head_2011);
    head_2011 = delPos_2011(head_2011, 2);

    printList(head_2011);
}
}

```

Pembahasan Class HapusDLL:

1. Deklarasi Kelas

public class HapusDLL_2511532011: Kelas publik yang berisi kumpulan metode statis untuk operasi penghapusan pada Doubly Linked List.

Kelas ini berfungsi sebagai utility class yang menyediakan metode delete head, delete tail, dan delete pada posisi tertentu dengan manipulasi pointer dua arah.

2. Metode delHead_2011 (Hapus Depan)

public static NodeDLL_2511532011 delHead_2011(...): Metode statis untuk menghapus node paling depan (head) dari DLL.

if (head_2011 == null) { return null; }: Guard clause. Jika list kosong, langsung kembalikan null karena tidak ada yang bisa dihapus.

NodeDLL_2511532011 temp_2011 = head_2011;; Simpan referensi head lama ke variabel temporary (opsional, untuk keperluan debug atau cleanup).

head_2011 = head_2011.next_2011;; Logika Forward. Pindahkan referensi head ke node berikutnya.

if (head_2011 != null) { head_2011.prev_2011 = null; }: Logika Backward (Khas DLL). Jika head baru tidak null, putuskan pointer prev-nya agar tidak lagi menunjuk ke node lama yang sudah dihapus. Ini menjaga konsistensi link dua arah.

return head_2011;; Kembalikan referensi head yang baru.

Kompleksitas: $O(1)$ karena hanya mengubah referensi pointer tanpa traversing.

3. Metode delLast (Hapus Belakang)

public static NodeDLL_2511532011 delLast(...): Metode statis untuk menghapus node paling akhir (tail) dari DLL.

if (head_2011 == null) { return null; }: Handle kasus list kosong.

if (head_2011.next_2011 == null) { return null; }: Handle kasus list hanya memiliki satu node. Setelah dihapus, list menjadi kosong.

NodeDLL_2511532011 curr_2011 = head_2011;; Inisialisasi pointer curr untuk traversing dari head.

while (curr_2011.next_2011 != null) { curr_2011 = curr_2011.next_2011; }: Loop traversing hingga menemukan node terakhir (tail) dimana next bernilai null.

if (curr_2011.prev_2011 != null) { curr_2011.prev_2011.next_2011 = null; }: Logika Penghapusan DLL. Putuskan pointer next dari node kedua-terakhir agar menunjuk ke null, sehingga tail terisolasi. Pointer prev dari tail tidak perlu diupdate karena tail akan dihapus.

return head_2011;; Kembalikan head asli karena posisi head tidak berubah.

Kompleksitas: $O(n)$ karena harus traversing seluruh list untuk mencapai tail.

4. Metode delPos_2011 (Hapus Posisi Tertentu)

public static NodeDLL_2511532011 delPos_2011(...): Metode untuk menghapus node pada posisi index tertentu (1-based index).

if (head_2011 == null) { return head_2011; }: Handle kasus list kosong.

NodeDLL_2511532011 curr_2011 = head_2011;; Inisialisasi pointer curr untuk traversing menuju node target.

for (int i = 1; curr_2011 != null && i < pos_2011; ++i) { curr_2011 = curr_2011.next_2011; }: Loop untuk bergerak hingga mencapai node di posisi target.

if (curr_2011 == null) { return head_2011; }: Validasi jika posisi melebihi panjang list.

Tiga Langkah Penyambungan Ulang (Khas DLL):

if (curr_2011.prev_2011 != null) { curr_2011.prev_2011.next_2011 = curr_2011.next_2011; }: Hubungkan node sebelumnya langsung ke node setelah target (lewati curr).

if (curr_2011.next_2011 != null) { curr_2011.next_2011.prev_2011 = curr_2011.prev_2011; }: Hubungkan node setelah target kembali ke node sebelumnya.

if (head_2011 == curr_2011) { head_2011 = curr_2011.next_2011; }: Jika node yang dihapus adalah head, update referensi head ke node berikutnya.

return head_2011;; Kembalikan referensi head yang mungkin telah berubah.

Kompleksitas: $O(n)$ karena perlu traversing untuk mencapai posisi target.

5. Metode printList (Cetak List)

public static void printList(NodeDLL_2511532011 head_2011): Metode untuk menampilkan isi DLL ke konsol.

NodeDLL_2511532011 curr_2011 = head_2011;; Inisialisasi pointer curr untuk traversing mulai dari head.

```
while (curr_2011 != null) { System.out.print(curr_2011.data_2011 + " ");  
curr_2011 = curr_2011.next_2011; };
```

 Loop mencetak setiap nilai data diikuti spasi, bergerak forward melalui pointer next.

System.out.println();: Pindah baris setelah selesai mencetak.

6. Metode main - Inisialisasi DLL Awal

```
NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(1);;
```

Membuat node pertama dengan nilai 1 sebagai head.

Rangkaian inisialisasi dengan pola: node.next = new Node(...);
node.next.prev = node; untuk setiap node berikutnya.

Penting: Setiap penambahan node selalu diikuti dengan setting pointer prev untuk membangun koneksi dua arah.

State Awal: DLL terbentuk: null <- [1] <-> [2] <-> [3] <-> [4] <-> [5] -> null.

7. Metode main - Cetak DLL Awal

```
System.out.print("DLL Awal: ");;
```

 Label output.

```
printList(head_2011);;
```

 Memanggil metode print untuk menampilkan: 1 2 3 4 5.

8. Metode main - Delete Head Pertama

```
head_2011 = delHead_2011(head_2011);;
```

 Panggil metode hapus head. Node dengan nilai 1 dihapus.

Proses: head bergeser ke 2, lalu 2.prev di-set ke null.

State: null <- [2] <-> [3] <-> [4] <-> [5] -> null.

```
printList(head_2011);;
```

 Cetak hasil: 2 3 4 5.

9. Metode main - Delete Head Kedua (Komentar Kode Tidak Sesuai)

```
head_2011 = delHead_2011(head_2011);;
```

 Catatan: Komentar kode menyebut "node terakhir" tetapi yang dipanggil adalah delHead_2011.

Node dengan nilai 2 dihapus dari depan.

State: null <- [3] <-> [4] <-> [5] -> null.

```
printList(head_2011);;
```

 Cetak hasil: 3 4 5.

10. Metode main - Delete Head Ketiga + Delete Position

```
head_2011 = delHead_2011(head_2011);;
```

 Hapus head lagi, node 3 terhapus.

State Sementara: null <- [4] <-> [5] -> null.

head_2011 = delPos_2011(head_2011, 2);: Hapus node di posisi ke-2 (1-based).

Proses: Posisi 2 saat ini adalah node 5. Lakukan bypass: 4.next = 5.next (null), dan karena 5.next null, tidak perlu update prev.

State Akhir: null <- [4] -> null.

printList(head_2011);: Cetak hasil akhir: 4.

```

package pekan6_2511532011;

public class PenulusuranDLL_2511532011 {
    static void forwardTraversal (NodeDLL_2511532011 head_2011) {
        NodeDLL_2511532011 curr_2011 = head_2011;
        while (curr_2011 != null) {
            System.out.print(curr_2011.data_2011 + " <-> ");
            curr_2011 = curr_2011.next_2011;
        }
        System.out.println();
    }

    static void backwardTraversal (NodeDLL_2511532011 tail_2011) {
        NodeDLL_2511532011 curr_2011 = tail_2011;
        while (curr_2011 != null) {
            System.out.print(curr_2011.data_2011 + " <-> ");
            curr_2011 = curr_2011.prev_2011;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(1);
        NodeDLL_2511532011 second_2011 = new NodeDLL_2511532011(2);
        NodeDLL_2511532011 third_2011 = new NodeDLL_2511532011(3);

        head_2011.next_2011 = second_2011;
        second_2011.prev_2011 = head_2011;
        second_2011.next_2011 = third_2011;
        third_2011.prev_2011 = second_2011;

        System.out.println("Penulusuran maju:");
        forwardTraversal(head_2011);
        System.out.println("Penulusuran mundur:");
        backwardTraversal(third_2011);
    }
}

```

Pembahasan Class PencarianDLL:

1. Deklarasi Kelas

public class PenelusuranDLL_2511532011: Kelas publik yang berisi metode statis untuk demonstrasi penelusuran (traversal) maju dan mundur pada Doubly Linked List.

2. Metode forwardTraversal (Penelusuran Maju)

static void forwardTraversal(NodeDLL_2511532011 head_2011): Metode statis untuk menelusuri list dari depan (head) ke belakang (tail).

NodeDLL_2511532011 curr_2011 = head_2011;; Inisialisasi pointer curr dimulai dari referensi head yang diterima sebagai parameter.

while (curr_2011 != null): Loop berjalan selama pointer curr belum mencapai akhir list (null).

System.out.print(curr_2011.data_2011 + " <-> "); Mencetak nilai data node saat ini diikuti pemisah " <-> " untuk memvisualisasikan koneksi dua arah.

curr_2011 = curr_2011.next_2011;; Logika Forward. Menggeser pointer curr ke node berikutnya melalui pointer next.

System.out.println(); Pindah baris setelah seluruh elemen selesai dicetak.

Kompleksitas: $O(n)$ karena harus mengunjungi setiap node sekali.

3. Metode backwardTraversal (Penelusuran Mundur)

static void backwardTraversal(NodeDLL_2511532011 tail_2011): Metode statis untuk menelusuri list dari belakang (tail) ke depan (head).

Perbedaan Utama: Parameter yang diterima adalah tail_2011 (node terakhir), bukan head.

NodeDLL_2511532011 curr_2011 = tail_2011;; Inisialisasi pointer curr dimulai dari referensi tail.

while (curr_2011 != null): Loop berjalan selama pointer curr belum mencapai awal list (null).

System.out.print(curr_2011.data_2011 + " <-> "); Mencetak nilai data node saat ini dengan pemisah yang sama.

curr_2011 = curr_2011.prev_2011;; Logika Backward (Khas DLL). Menggeser pointer curr ke node sebelumnya melalui pointer prev. Inilah keunggulan DLL yang tidak dimiliki SLL.

System.out.println(); Pindah baris setelah traversal selesai.

Kompleksitas: $O(n)$ karena harus mengunjungi setiap node sekali.

4. Metode main - Inisialisasi Node

```
NodeDLL_2511532011 head_2011 = new NodeDLL_2511532011(1);;
```

Membuat node pertama dengan nilai 1 yang akan menjadi head.

```
NodeDLL_2511532011 second_2011 = new NodeDLL_2511532011(2);;
```

Membuat node kedua dengan nilai 2.

```
NodeDLL_2511532011 third_2011 = new NodeDLL_2511532011(3);;
```

Membuat node ketiga dengan nilai 3 yang akan menjadi tail.

Metode main - Membangun Link Dua Arah

```
head_2011.next_2011 = second_2011;; Hubungkan next dari node 1 ke  
node 2.
```

```
second_2011.prev_2011 = head_2011;; Penting: Hubungkan prev dari  
node 2 kembali ke node 1. Ini membangun koneksi dua arah pertama.
```

```
second_2011.next_2011 = third_2011;; Hubungkan next dari node 2 ke  
node 3.
```

```
third_2011.prev_2011 = second_2011;; Hubungkan prev dari node 3  
kembali ke node 2.
```

```
State Akhir DLL: null <- [1] <-> [2] <-> [3] -> null
```

5. Metode main - Eksekusi Forward Traversal

```
System.out.println("Penelusuran maju:");; Mencetak label untuk  
penelusuran maju.
```

```
forwardTraversal(head_2011);; Memanggil metode traversal maju dengan  
parameter head (node 1).
```

Alur Eksekusi:

```
Iterasi 1: curr = 1, cetak "1 <-> ", curr = curr.next (ke 2)
```

```
Iterasi 2: curr = 2, cetak "2 <-> ", curr = curr.next (ke 3)
```

```
Iterasi 3: curr = 3, cetak "3 <-> ", curr = curr.next (null)
```

Loop berakhir

```
Output: 1 <-> 2 <-> 3 <->
```

6. Metode main - Eksekusi Backward Traversal

```
System.out.println("Penelusuran mundur:");; Mencetak label untuk  
penelusuran mundur.
```

backwardTraversal(third_2011);: Memanggil metode traversal mundur dengan parameter tail (node 3).

Alur Eksekusi:

Iterasi 1: curr = 3, cetak "3 <-> ", curr = curr.prev (ke 2)

Iterasi 2: curr = 2, cetak "2 <-> ", curr = curr.prev (ke 1)

Iterasi 3: curr = 1, cetak "1 <-> ", curr = curr.prev (null)

Loop berakhir

Output: 3 <-> 2 <-> 1 <->

2.2 Tugas Struktur Data

```
package pekan6_2511532011;

public class lagu_2511532011 {
    String judul_2011;
    String penyanyi_2011;
    lagu_2511532011 next_2011;
    lagu_2511532011 prev_2011;

    public lagu_2511532011(String j_2011, String p_2011)
    {
        this.judul_2011 = j_2011;
        this.penyanyi_2011 = p_2011;
        this.next_2011 = null;
        this.prev_2011 = null;
    }

    public void setJ_2011(String j_2011) {
        this.judul_2011 = j_2011;
    }

    public void setP_2011(String p_2011) {
        this.penyanyi_2011 = p_2011;
    }

    public String getj_2011() {
        return judul_2011;
    }

    public String getp_2011() {
        return penyanyi_2011;
    }
}
```

Pembahasan Class Lagu:

1. Deklarasi Kelas

public class lagu_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai cetak biru (blueprint) untuk objek data lagu.

Kelas ini dirancang sebagai node dalam Doubly Linked List, sehingga memiliki atribut data dan pointer dua arah.

2. Atribut Data: Judul Lagu

String judul_2011;; Variabel instance bertipe String untuk menyimpan judul atau nama lagu.

Merupakan informasi utama yang diidentifikasi oleh pengguna dalam playlist.

3. Atribut Data: Nama Penyanyi

String penyanyi_2011;; Variabel instance bertipe String untuk menyimpan nama artis atau penyanyi lagu.

Melengkapi informasi lagu untuk identifikasi yang lebih spesifik.

4. Atribut Pointer Next (Forward Link)

lagu_2511532011 next_2011;; Variabel instance bertipe referensi ke kelas lagu_2511532011 itu sendiri.

Berfungsi sebagai pointer forward yang menghubungkan node lagu saat ini ke lagu berikutnya dalam playlist.

Memungkinkan traversing dari awal ke akhir playlist (fitur "Next Song").

5. Atribut Pointer Prev (Backward Link)

lagu_2511532011 prev_2011;; Variabel instance bertipe referensi ke kelas lagu_2511532011 itu sendiri.

Perbedaan Utama dengan SLL: Pointer prev memungkinkan navigasi mundur, yaitu dari lagu saat ini ke lagu sebelumnya.

Memungkinkan traversing dari akhir ke awal playlist (fitur "Previous Song") - keunggulan utama DLL.

6. Konstruktor Kelas

public lagu_2511532011(String j_2011, String p_2011): Konstruktor publik yang menerima dua parameter String untuk inisialisasi objek lagu baru.

Parameter j_2011: Nilai String untuk judul lagu.

Parameter p_2011: Nilai String untuk nama penyanyi.

7. Inisialisasi Atribut Data dalam Konstruktor

this.judul_2011 = j_2011;; Assign nilai parameter j_2011 ke variabel instance judul_2011.

`this.penyanyi_2011 = p_2011;` Assign nilai parameter `p_2011` ke variabel instance `penyanyi_2011`.

Keyword `this` digunakan untuk membedakan antara atribut kelas dengan parameter metode yang memiliki nama sama.

8. Inisialisasi Pointer Next dalam Konstruktor

`this.next_2011 = null;` Mengassign nilai `null` ke atribut pointer `next_2011`.

Inisialisasi `null` menandakan bahwa node lagu baru ini belum terhubung ke lagu berikutnya.

Dalam DLL, node dengan `next = null` biasanya merepresentasikan lagu terakhir dalam playlist (tail).

9. Inisialisasi Pointer Prev dalam Konstruktor

`this.prev_2011 = null;` Mengassign nilai `null` ke atribut pointer `prev_2011`.

Inisialisasi `null` menandakan bahwa node lagu baru ini belum terhubung ke lagu sebelumnya.

Dalam DLL, node dengan `prev = null` biasanya merepresentasikan lagu pertama dalam playlist (head).

10. Metode Setter `setJ_2011`

`public void setJ_2011(String j_2011):` Metode publik untuk mengubah/update nilai judul lagu setelah objek dibuat.

`this.judul_2011 = j_2011;` Assign nilai parameter baru ke atribut `judul_2011`.

Memungkinkan mutasi data judul tanpa perlu membuat objek lagu baru.

11. Metode Setter `setP_2011`

`public void setP_2011(String p_2011):` Metode publik untuk mengubah/update nilai nama penyanyi.

`this.penyanyi_2011 = p_2011;` Assign nilai parameter baru ke atribut `penyanyi_2011`.

Berguna jika ada koreksi nama artis atau update informasi metadata lagu.

12. Metode Getter `getj_2011`

`public String getj_2011():` Metode publik untuk mengakses nilai judul lagu dari luar kelas.

return judul_2011;; Mengembalikan nilai String dari atribut judul_2011.

Menerapkan prinsip enkapsulasi: data diakses melalui metode publik terkontrol.

13. Metode Getter getp_2011

public String getp_2011(): Metode publik untuk mengakses nilai nama penyanyi.

return penyanyi_2011;; Mengembalikan nilai String dari atribut penyanyi_2011.

Digunakan oleh metode display dan search untuk menampilkan informasi lagu.

```

package pekan6_2511532011;
import java.util.Scanner;

public class musik_2511532011 {

    public static lagu_2511532011
    tambahLagu_2011(lagu_2511532011 l_2011, String j_2011,
    String p_2011) {
        lagu_2511532011 lg_2011 = new
    lagu_2511532011(j_2011, p_2011);

        if (l_2011 == null) return lg_2011;

        lagu_2511532011 curr_2011 = l_2011;
        while (curr_2011.next_2011 != null) {
            curr_2011 = curr_2011.next_2011;
        }

        curr_2011.next_2011 = lg_2011;
        lg_2011.prev_2011 = curr_2011;
        return l_2011;
    }

    public static lagu_2511532011
    hapusLaguAwal_2011(lagu_2511532011 l_2011) {
        if (l_2011 == null) {
            return null;
        }

        lagu_2511532011 temp_2011 = l_2011;
        l_2011 = l_2011.next_2011;

        if (l_2011 != null) {
            l_2011.prev_2011 = null;
        }

        temp_2011.next_2011 = null;
        return l_2011;
    }

    public static void tampilMaju_2011(lagu_2511532011
    l_2011) {
        if (l_2011 == null) {
            System.out.println("Playlist kosong.");
            return;
        }
    }
}

```

```

        lagu_2511532011 curr_2011 = l_2011;
        while (curr_2011 != null) {
            System.out.print(curr_2011.getj_2011() + " " +
curr_2011.getp_2011() + " <-> ");
            curr_2011 = curr_2011.next_2011;
        }
        System.out.println();
    }

    public static void tampilMundur_2011(lagu_2511532011
l_2011) {
        if (l_2011 == null) {
            System.out.println("Playlist kosong.");
            return;
        }

        lagu_2511532011 curr_2011 = l_2011;
        while (curr_2011.next_2011 != null) {
            curr_2011 = curr_2011.next_2011;
        }

        while (curr_2011 != null) {
            System.out.print(curr_2011.getj_2011() + " " +
curr_2011.getp_2011() + " <-> ");
            curr_2011 = curr_2011.prev_2011;
        }
        System.out.println();
    }

    public static boolean cariLagu_2011(lagu_2511532011
l_2011, String j_2011) {
        lagu_2511532011 curr_2011 = l_2011;
        while (curr_2011 != null) {
            if
(curr_2011.judul_2011.equalsIgnoreCase(j_2011)) {
                System.out.println("Judul Lagu: " +
curr_2011.getj_2011());
                System.out.println("Penyanyi : " +
curr_2011.getp_2011());
                return true;
            }
            curr_2011 = curr_2011.next_2011;
        }
        System.out.println("tidak ada lagu yang anda cari di
playlist");
        return false;
    }
}

```

```

public static void main(String[] args) {
    lagu_2511532011 item_2011 = null;
    Scanner input_2011 = new Scanner(System.in);
    int pilihan_2011 = 0;

    do {
        System.out.println("\n=== playlist Musik
Nim:2511532011 ===");
        System.out.println("1. Tambah lagu");
        System.out.println("2. Hapus lagu awal");
        System.out.println("3. lihat playlist (maju)");
        System.out.println("4. lihat playlist
(mundur)");
        System.out.println("5. Cari lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        if (input_2011.hasNextInt()) {
            pilihan_2011 = input_2011.nextInt();
            input_2011.nextLine();
        } else {
            System.out.println("Input tidak valid.
Masukkan angka.");
            input_2011.nextLine();
            continue;
        }

        switch (pilihan_2011) {
            case 1:
                System.out.print("Masukkan Judul lagu:
");
                String j_2011 = input_2011.nextLine();
                System.out.print("Masukkan Nama
Penyanyi: ");
                String p_2011 = input_2011.nextLine();

                item_2011 = tambahLagu_2011(item_2011,
j_2011, p_2011);
                System.out.println("lagu berhasil
ditambahkan");
                break;

            case 2:
                item_2011 =
hapusLaguAwal_2011(item_2011);
                break;

```

```

        case 3:
            tampilMaju_2011(item_2011);
            break;

        case 4:
            tampilMundur_2011(item_2011);
            break;

        case 5:
            System.out.print("Masukkan judul lagu
yang dicari: ");
            String i_2011 = input_2011.nextLine();
            cariLagu_2011(item_2011, i_2011);
            break;

        case 6:
            System.out.println("Program selesai");
            break;

        default:
            System.out.println("Pilihan tidak
valid.");
            break;
    }

    } while (pilihan_2011 != 6);

    input_2011.close();
}
}

```

Pembahasan Class Musik:

1. Deklarasi Kelas

public class musik_2511532011: Kelas publik yang berfungsi sebagai driver utama untuk sistem manajemen playlist musik berbasis Doubly Linked List.

2. Metode tambahLagu_2011 (Tambah Lagu di Akhir)

public static lagu_2511532011 tambahLagu_2011(...): Metode statis untuk menambahkan lagu baru ke akhir playlist (operasi insert tail pada DLL).

lagu_2511532011 lg_2011 = new lagu_2511532011(j_2011, p_2011);::

Membuat node lagu baru dengan judul dan penyanyi yang diinput.

if (l_2011 == null) return lg_2011;; Handle kasus playlist kosong. Node baru langsung menjadi head dan dikembalikan.

lagu_2511532011 curr_2011 = l_2011;; Inisialisasi pointer curr untuk traversing dari head.

while (curr_2011.next_2011 != null) { curr_2011 = curr_2011.next_2011; }: Loop traversing hingga menemukan node terakhir (tail) dimana next bernilai null.

curr_2011.next_2011 = lg_2011;; Logika Forward Link. Hubungkan tail saat ini ke node lagu baru.

lg_2011.prev_2011 = curr_2011;; Logika Backward Link (Khas DLL). Hubungkan pointer prev lagu baru ke tail lama. Ini memastikan koneksi dua arah tetap utuh.

return l_2011;; Kembalikan referensi head karena posisi head tidak berubah.

Kompleksitas: $O(n)$ karena harus traversing seluruh list untuk mencapai tail.

3. Metode hapusLaguAwal_2011 (Hapus Lagu Pertama)

public static lagu_2511532011 hapusLaguAwal_2011(...): Metode untuk menghapus lagu terdepan dari playlist (operasi delete head pada DLL).

if (l_2011 == null) { return null; }: Handle kasus playlist kosong.

lagu_2511532011 temp_2011 = l_2011;; Simpan referensi head lama ke variabel temporary (untuk keperluan cleanup atau debug).

l_2011 = l_2011.next_2011;; Logika Forward. Pindahkan referensi head ke node berikutnya.

if (l_2011 != null) { l_2011.prev_2011 = null; }: Logika Backward (Khas DLL). Jika head baru tidak null, putuskan pointer prev-nya agar tidak lagi menunjuk ke node lama yang sudah dihapus.

temp_2011.next_2011 = null;; Putuskan referensi next dari node lama sebagai praktik good practice untuk membantu garbage collector.

return l_2011;; Kembalikan referensi head yang baru.

Kompleksitas: $O(1)$ karena hanya mengubah referensi pointer tanpa traversing.

4. Metode tampilMaju_2011 (Display Forward)

public static void tampilMaju_2011(...): Metode untuk menampilkan isi playlist dari awal ke akhir.

if (l_2011 == null) { System.out.println("Playlist kosong."); return; }:
Handle kasus playlist kosong.

lagu_2511532011 curr_2011 = l_2011;; Inisialisasi pointer curr untuk traversing mulai dari head.

while (curr_2011 != null): Loop berjalan selama masih ada node yang belum diakses.

System.out.print(curr_2011.getj_2011() + " " + curr_2011.getp_2011() + "<-> "); Mencetak judul dan penyanyi lagu diikuti pemisah "<-> " untuk visualisasi koneksi dua arah.

curr_2011 = curr_2011.next_2011;; Geser pointer ke node berikutnya melalui next.

System.out.println(); Pindah baris setelah selesai mencetak.

5. Metode tampilMundur_2011 (Display Backward)

public static void tampilMundur_2011(...): Metode untuk menampilkan isi playlist dari akhir ke awal (fitur unik DLL).

if (l_2011 == null) { System.out.println("Playlist kosong."); return; }:
Handle kasus playlist kosong.

lagu_2511532011 curr_2011 = l_2011;; Inisialisasi pointer dari head.

while (curr_2011.next_2011 != null) { curr_2011 = curr_2011.next_2011; }; Langkah Persiapan. Traversing forward terlebih dahulu untuk menemukan tail (node terakhir).

while (curr_2011 != null): Loop traversal mundur dimulai dari tail.

System.out.print(curr_2011.getj_2011() + " " + curr_2011.getp_2011() + "<-> "); Cetak judul dan penyanyi dengan pemisah yang sama.

curr_2011 = curr_2011.prev_2011;; Logika Backward (Khas DLL). Geser pointer ke node sebelumnya melalui prev.

System.out.println(); Pindah baris setelah traversal selesai.

6. Metode cariLagu_2011 (Search by Title)

public static boolean cariLagu_2011(...): Metode untuk mencari lagu berdasarkan judul dalam playlist.

lagu_2511532011 curr_2011 = l_2011;; Inisialisasi pointer traversing dari head.

while (curr_2011 != null): Loop linear search melalui seluruh elemen list.

if (curr_2011.judul_2011.equalsIgnoreCase(j_2011)): Logika Pencarian. Membandingkan judul lagu dengan input menggunakan equalsIgnoreCase agar pencarian tidak sensitif terhadap huruf besar/kecil.

Jika ditemukan: Cetak detail lagu (judul dan penyanyi) menggunakan getter, lalu return true untuk menghentikan pencarian (early return).

curr_2011 = curr_2011.next_2011;; Lanjut ke node berikutnya jika tidak cocok.

System.out.println("tidak ada lagu yang anda cari di playlist"); return false;; Jika loop selesai tanpa menemukan kecocokan, tampilkan pesan tidak ditemukan.

7. Metode main - Inisialisasi

lagu_2511532011 item_2011 = null;; Inisialisasi referensi head playlist dengan null, menandakan playlist awal kosong.

Scanner input_2011 = new Scanner(System.in);; Aktifkan scanner untuk input konsol.

int pilihan_2011 = 0;; Variabel penampung pilihan menu dari pengguna.

Metode main - Loop Menu Interaktif

do { ... } while (pilihan_2011 != 6);; Struktur loop yang memastikan menu tampil minimal sekali dan berulang hingga pengguna memilih opsi 6 (Keluar).

Blok System.out.println(...): Mencetak header dan opsi menu (1-6) untuk panduan pengguna.

Validasi Input Pengguna

if (input_2011.hasNextInt()): Validasi tipe input. Mencegah program crash jika pengguna memasukkan huruf/simbol.

pilihan_2011 = input_2011.nextInt(); input_2011.nextLine();; Baca angka pilihan dan konsumsi karakter newline tersisa agar input string berikutnya tidak terlewat.

Blok else: Menangani input tidak valid dengan pesan error dan continue untuk mengulang loop.

Switch Case - Menu 1: Tambah Lagu

case 1:: Prompt input judul lagu dan nama penyanyi menggunakan input_2011.nextLine().

item_2011 = tambahLagu_2011(item_2011, j_2011, p_2011);; Panggil metode insert tail. Penting: Hasil return harus di-assign kembali ke item_2011 karena head bisa berubah (saat playlist awal kosong).

Cetak konfirmasi "lagu berhasil ditambahkan".

Switch Case - Menu 2: Hapus Lagu Awal

case 2:: item_2011 = hapusLaguAwal_2011(item_2011);; Panggil metode delete head. Head bergeser ke lagu berikutnya, lagu lama terhapus dari playlist.

Switch Case - Menu 3 & 4: Tampilkan Playlist

case 3:: tampilMaju_2011(item_2011);; Tampilkan playlist dari awal ke akhir (forward traversal).

case 4:: tampilMundur_2011(item_2011);; Tampilkan playlist dari akhir ke awal (backward traversal) - fitur unik DLL.

Switch Case - Menu 5: Cari Lagu

case 5:: Prompt input judul pencarian, lalu cariLagu_2011(item_2011, i_2011);; Jalankan linear search dan tampilkan hasil jika ditemukan.

Switch Case - Menu 6 & Default

case 6:: Cetak pesan terminasi program.

default:: Tangani angka di luar range 1-6 dengan pesan "Pilihan tidak valid".

Terminasi Program

input_2011.close();; Menutup resource scanner setelah loop selesai.

Praktik terbaik untuk mencegah resource leak.

Output:

terminated> musik_2511532011 [Java Application] C:\Program Files\Java\jdk-2

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
 2. Hapus lagu awal
 3. lihat playlist (maju)
 4. lihat playlist (mundur)
 5. Cari lagu
 6. Keluar
- Pilihan: 2

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
 2. Hapus lagu awal
 3. lihat playlist (maju)
 4. lihat playlist (mundur)
 5. Cari lagu
 6. Keluar
- Pilihan: 3

b 002 <-> c 003 <->

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar



24°C
Berawan



S

<terminated> Musik_2511532011 [Java Application] C:\Program Files\Java\jdk

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 4

c 003 <-> b 002 <-> a 001 <->

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu yang dicari: a

Judul Lagu: a

Penyanyi : 001

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)



<terminated> musik_2511532011 [Java Application] C:\Program Files\Java\jdk-

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 1

Masukkan Judul lagu: c

Masukkan Nama Penyanyi: 003

lagu berhasil ditambahkan

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 3

a 001 <-> b 002 <-> c 003 <->

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)

<terminated> Musik_2511532011 [Java Application] C:\Program Files\Java\jdk

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 1

Masukkan Judul lagu: a

Masukkan Nama Penyanyi: 001

lagu berhasil ditambahkan

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar

Pilihan: 1

Masukkan Judul lagu: b

Masukkan Nama Penyanyi: 002

lagu berhasil ditambahkan

=== playlist Musik Nim:2511532011 ===

1. Tambah lagu

BAB III KESIMPULAN

3.1 Kesimpulan

ADT LinkedList merupakan struktur data linear berbasis node yang saling terhubung melalui referensi, sehingga memungkinkan pengelolaan data secara dinamis tanpa memerlukan alokasi memori yang bersebelahan. Dalam bahasa pemrograman Java, LinkedList diimplementasikan sebagai doubly-linked list yang menyediakan metode utama seperti add(), remove(), get(), dan set() untuk manipulasi elemen secara terstruktur. Mekanisme kerja LinkedList mengutamakan efisiensi pada operasi penyisipan dan penghapusan karena tidak memerlukan penggeseran elemen, meskipun hal ini mengakibatkan kompleksitas waktu linear saat melakukan akses acak. Karakteristik tersebut menjadikan LinkedList solusi ideal untuk berbagai skenario komputasi yang membutuhkan fleksibilitas data, seperti manajemen riwayat navigasi, representasi graf melalui adjacency list, serta sistem playlist dan cache dinamis. Melalui praktikum ini, pemahaman konseptual mengenai struktur data non-kontigu serta kemampuan teknis dalam memanfaatkan metode LinkedList pada Java telah tercapai secara optimal sebagai fondasi pengembangan aplikasi yang efisien.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:29-april-2026]

